

Privacy-Preserving Social Recommendation: Privacy Leakage and Countermeasure

Yuyue Chen
Harbin Institute of Technology, Shenzhen
Shenzhen, China
chenyuyue@stu.hit.edu.cn

Peng Yang
The University of Hong Kong
Hong Kong, China
Harbin Institute of Technology, Shenzhen
Shenzhen, China
stuyangpeng@gmail.com

Zoe Lin Jiang*
Harbin Institute of Technology, Shenzhen
Shenzhen, China
Guangdong Key Laboratory of New Security and Intelligence Technology
Shenzhen, China
zoeljiang@hit.edu.cn

Wenhao Wu
Harbin Institute of Technology, Shenzhen
Shenzhen, China
cswenhao@stu.hit.edu.cn

Junbin Fang
Jinan University
Guangzhou, China
tjunbinfang@jnu.edu.cn

Xuan Wang
Harbin Institute of Technology, Shenzhen
Shenzhen, China
wangxuan@cs.hitsz.edu.cn

Chuanyi Liu
Harbin Institute of Technology, Shenzhen
Shenzhen, China
liuchuanyi@hit.edu.cn

Abstract

Social recommendation systems generally utilize two types of data, user-item interaction matrices (**R**) from rating platform (P_0), and user-user social graphs (**S**) from social platform (P_1). Considering user privacy that neither **R** nor **S** can be directly shared, Chen et al. introduced the Secure Social Recommendation (SeSoRec) framework with the Secret Sharing-based Matrix Multiplication (SSMM) protocol. However, we find that the leakage of intermeidate information introduced by SSMM will eventually lead to the leakage of **S** to P_0, which challenges the privacy guarantees of SeSoRec.

This work firstly identifies that the claimed "innocuous" leakage in SeSoRec originates from reusing the same One-Time Pad key during two randomization phases in SSMM, with formal proof that SSMM violates semi-honest security. Secondly, this work proposes the Two-Time Pad Attack with two reconstruction algorithms to evaluate the severity of the leakage. The Two-Time Pad Attack can extract the column-wise sum of matrices $\mathbf{A}_{c_even}$ and $\mathbf{A}_{c_odd}$, and the row-wise difference of matrices $\mathbf{B}_{r_even}$ and $\mathbf{B}_{r_odd}$, where such matrices are closely related to **R** or **S**. The Sparse Matrix Reconstruction (SMR) algorithm can achieve 99.35%, 83.83%, and 77.14% reconstruction rates for non-zero entries in **S** on FilmTrust, Epinions, and Douban datasets, respectively. The Grayscale Image Reconstruction (GIR) algorithm can successfully recover MNIST image contours. Thirdly, when the number of columns/rows of the input matrix $\mathbf{A}/\mathbf{B}$ in SSMM is odd (requiring zero-padding to an even dimension), this work proposes the Zero-Padding Attack which can directly expose the last column/row of $\mathbf{A}/\mathbf{B}$. Finally, this work proposes the Privacy-Preserving Matrix Multiplication (PPMM) protocol with experimental demonstration as a replacement for SSMM, which eliminates such leakage while maintaining efficiency.

CCS Concepts

• **Information systems** → **Social recommendation**; • **Security and privacy** → *Privacy-preserving protocols*; **Cryptanalysis and other attacks**.

Keywords

Social Recommendation Systems, Privacy-Preserving Social Recommendation, Secret Sharing

ACM Reference Format:
Yuyue Chen, Peng Yang, Zoe Lin Jiang, Wenhao Wu, Junbin Fang, Xuan Wang, and Chuanyi Liu. 2025. Privacy-Preserving Social Recommendation: Privacy Leakage and Countermeasure. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25), September 22–26, 2025, Prague, Czech Republic.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3705328.3748051

*Corresponding author: zoeljiang@hit.edu.cn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RecSys '25, Prague, Czech Republic

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1364-4/25/09
https://doi.org/10.1145/3705328.3748051

1 Introduction

Social recommendation systems rely on the synergistic use of user-item interaction data and user-user social relationships to generate personalized recommendations. However, in real-world applications, these two critical data types are typically segregated due to platform specialization and privacy constraints. E-commerce platforms (e.g., Amazon) with extensive user-item interaction records usually lack comprehensive social network data. Conversely, social media platforms (e.g., Facebook) with rich social graphs are legally and ethically constrained from sharing their sensitive information with third parties. Such data separation poses a challenge of how to effectively enhance recommendation accuracy through social relationships without direct accessing to the raw social data that becomes increasingly critical as users demand both highly personalized and privacy-preserving services.

The problem of secure social recommendation can be considered as a Multi-Party Computation (MPC) problem. MPC allows multiple parties to jointly compute a function without revealing their private inputs [8, 22]. The fundamental MPC protocols include garbled circuits [22] and secret sharing [8]. In the context of social recommendation systems, the rating platform and the social platform act as two parties in MPC, with the social recommendation system being the function they need to compute together. Chen et al. [2] proposed the Secure Social Recommendation (SeSoRec) framework, with the Secret Sharing-based Matrix Multiplication (SSMM) protocol. However, the efficiency of SSMM comes at the cost of leaking information, specifically the sum of the even and odd columns of the left input matrix $\mathbf{A} \in \mathbb{Z}_{2^l}^{d_1 \times d_2}$ (i.e., $\mathbf{A}_{c_even} + \mathbf{A}_{c_odd}$) and the difference between the even and odd rows of the right input matrix $\mathbf{B} \in \mathbb{Z}_{2^l}^{d_2 \times d_3}$ (i.e., $\mathbf{B}_{r_even} - \mathbf{B}_{r_odd}$).

This work firstly identifies that the fundamental cause of information leakage in SSMM is the reuse of the same one-time pad key ($\mathbf{R}_{c_odd}$) in the two randomization processes, masking the input matrix $\mathbf{A}$ with the random matrix $\mathbf{R}$ ($\mathbf{R} + \mathbf{A}$) and self-masking of the random matrix ($\mathbf{R}_{c_even} + \mathbf{R}_{c_odd}$, where both $\mathbf{R}_{c_even}$ and $\mathbf{R}_{c_odd}$ are parts of $\mathbf{R}$). It demonstrates that SSMM does not satisfy semi-honest security as defined by the real-ideal simulation paradigm.

Further, to obtain the leaked information and reconstruct the private information, this work proposes the Two-time Pad Attack to obtain leaked information from SSMM (i.e., $\mathbf{A}_{c_even} + \mathbf{A}_{c_odd}$ and $\mathbf{B}_{r_even} - \mathbf{B}_{r_odd}$). Then introduce the Sparse Matrix Reconstruction (SMR) algorithm, demonstrating that using the leaked information ($\mathbf{B}_{r_even} - \mathbf{B}_{r_odd}$) as input reveals 99.35%, 83.83%, and 77.14% of the non-zero entries of the social matrices in the commonly used recommendation datasets FilmTrust, Epinions, and Douban, respectively. Additionally, propose the Grayscale Image Reconstruction (GIR) algorithm to demonstrate that the original image contour information on the MNIST dataset can be reconstructed using the leaked information $\mathbf{A}_{c_even} + \mathbf{A}_{c_odd}$ and $\mathbf{B}_{r_even} - \mathbf{B}_{r_odd}$. Subsequently, following the instruction of SSMM to pad the last column or row with 0s when the number of columns in the left input matrix or rows in the right input matrix is odd, introduce the Zero-Padding Attack. The attack reveals the last column of the left input matrix ($\mathbf{A}[: d_2 - 1]$) and the last row of the right input matrix ($\mathbf{B}[d_2 - 1 :]$).

Finally, to construct a secure framework, this paper proposes a Privacy-Preserving Matrix Multiplication (PPMM) protocol with provable semi-honest security. By replacing the SSMM protocol with the PPMM protocol in the SeSoRec framework, we implement an efficient and secure social recommendation system (SeSoRec-Sec). According to the characteristic that the same input is used multiple times when the PPMM is called in SeSoRec-Sec, by leveraging the technology of correlated matrix multiplication triples, the online communication overhead of SeSoRec-Sec is reduced from $6\ell(d_1d_2 + d_2d_3)$ to $2\ell(d_1d_2 + 3d_2d_3)$. Experimental results show that the above attacks are ineffective under this system, and the accuracy of SeSoRec-Sec is almost the same as that of SeSoRec.

Our Contributions.

- Identify and trace the information leakage in SeSoRec [2] conceptually.
- Propose the Two-time Pad Attack with the Sparse Matrix Reconstruction (SMR) and the Grayscale Image Reconstruction (GIR) algorithms, and the Zero-Padding Attack. Experimental results demonstrate the serious leakage of the original private information.
- Propose a privacy-preserving matrix multiplication (PPMM) protocol as a countermeasure to enhance the security of SeSoRec and utilize the correlated matrix multiplication triples to further reduce the online communication overhead.

2 Related Work

Privacy-preserving social recommendation systems can enhance user experience while protecting the privacy of user-item interaction data and user-user social information. Currently, most research implements privacy-preserving recommendations using Differential Privacy (DP)[5] and cryptographic methods[7, 22]. Methods based on Differential Privacy generally offer lower computational accuracy and weaker privacy protection compared to cryptographic methods. Cryptographic-based privacy-preserving recommendation systems utilize Homomorphic Encryption (HE)[7] and Secure Multi-Party Computation (MPC)[22] to ensure the privacy of social data. Homomorphic Encryption provides high security but suffers from poor performance and generality. Secure Multi-Party Computation not only offers high security but also better practical efficiency. In recent years, many studies [2, 3, 18] have applied MPC to privacy-preserving recommendation systems. There are various methods for building recommendation systems, and when constructing privacy-preserving recommendation systems using MPC, different specific protocols need to be designed according to the method. This paper focuses on factorization-based social recommendation systems. Chen et al. [2] designed a lightweight MPC protocol for cross-platform item computation in social recommendations, overcoming the high communication costs of MPC through matrix block multiplication. However, our analysis shows that Chen et al.'s protocol is insecure and leads to privacy leakage. Cui et al.[3] extended the work of [2] by leveraging the sparsity of the data and employed Homomorphic Encryption and Private Information Retrieval (PIR) to create a communication-efficient privacy-preserving recommendation protocol. However, it depends on the strong assumption that both parties know the positions of

the non-zero values in the sparse matrix, or that the number of zeros is publicly known.

3 Preliminaries

Notations. A set is denoted by X, and the set of integers is denoted by $\mathbb{Z}$. The ring of integers modulo 2^ℓ is denoted by $\mathbb{Z}_{2^\ell}$. Scalars, vectors, and matrices are denoted by lowercase letters (e.g., x), lowercase bold (e.g., $\mathbf{x}$), and uppercase bold (e.g., $\mathbf{X}$), respectively. The entry in the i-th row and the j-th column of a matrix $\mathbf{X}$ is denoted by $\mathbf{X}[i, j]$. $\mathbf{X}_{c_odd}$ and $\mathbf{X}_{c_even}$ represent the matrices composed of all odd columns and even columns of the matrix $\mathbf{X}$, respectively. $\mathbf{X}_{r_odd}$ and $\mathbf{X}_{r_even}$ represent the matrices composed of all odd rows and even rows of the matrix $\mathbf{X}$, respectively.

3.1 Social Recommendation and SeSoRec

One of the most popular methods in recommendation systems is factorization-based recommendation [13, 16]. This method decomposes a user-item rating (or other interaction) matrix into a user-latent matrix and an item-latent matrix. Distinct from traditional recommendation algorithms that solely consider the user-item rating matrix, social recommendation algorithms incorporate users' social network information to enhance the recommendation system. Social recommendation can be formalized as a basic decomposition model plus a social information model based on the assumption that related users tend to have similar preferences. Most existing social factorization models possess the following objective function:

$$\min_{\mathbf{U_i},\mathbf{V_j}} \sum_{i=1}^{I}\sum_{j=1}^{J} \frac{1}{2} I_{ij} \left(r_{ij} - \mathbf{U_i}^T \mathbf{V_j}\right)^2 + \gamma \sum_{i=1}^{I}\sum_{f=1}^{I} \frac{1}{2} s_{if} \left\|\mathbf{U_i} - \mathbf{U_f}\right\|_F^2 \quad (1)$$

where I_{ij} is an indicator function, equal to 1 if there is an existing user-item interaction pair, otherwise it is 0. $(r_{ij} - \mathbf{U_i}^T \mathbf{V_j})^2$ is the loss of the fundamental decomposition model that constrains the relationship between the real ratings and the predicted ratings. $\|\mathbf{U_i} - \mathbf{U_f}\|_F^2$ is the constraint for users with social relationships, γ controls the strength of the social constraints, and $||\cdot||_F^2$ is the Frobenius norm.

Considering the scenario that the rating platform P_0 with user-item rating matrix wants to construct a recommendation system with the help of the social platform P_1 with user-user rating matrix without leaking private information from each other. Specifically, in order to prevent the private input s_{if} of the social platform from being leaked during the solution process using stochastic gradient descent, the objective function is modified in the social decomposition model in SeSoRec [2]:

$$\begin{aligned}\min_{\mathbf{U_B},\mathbf{V_B}} \mathcal{L} = & \frac{1}{2} \left\|\mathbf{I_B} \circ \left(\mathbf{R_B} - \mathbf{U_B}^T \mathbf{V_B}\right)\right\|_F^2 \\ & + \frac{\gamma}{2} \mathrm{SUM}\left(\mathbf{D_B} \circ \left(\mathbf{U_B}^T \mathbf{U_B}\right)\right) - \gamma \mathrm{SUM}\left(\mathbf{S_B} \circ \left(\mathbf{U_B}^T \mathbf{U}\right)\right) \\ & + \frac{\gamma}{2} \mathrm{SUM}\left(\mathbf{E} \circ \left(\mathbf{U}^T \mathbf{U}\right)\right) + \frac{\lambda}{2} \left(\|\mathbf{U_B}\|_F^2 + \|\mathbf{V_B}\|_F^2\right)\end{aligned} \quad (2)$$

where $\circ$ denotes the Hadamard product, and SUM represents the summation of all elements within the matrix.

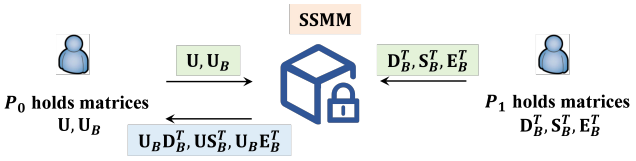


Figure 1: SeSoRec with SSMM.

The local minimum of the objective function can be found by:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{U_B}} = & -\mathbf{V_B} \left(\left(\mathbf{R_B} - \mathbf{U_B}^T \mathbf{V_B}\right)^T \circ \mathbf{I_B}\right) \\ & + \frac{\gamma}{2} \mathbf{U_B} \mathbf{D_B}^T - \gamma \mathbf{U} \mathbf{S_B}^T + \frac{\gamma}{2} \mathbf{U_B} \mathbf{E_B}^T + \lambda \mathbf{U_B}\end{aligned} \quad (3)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{V_B}} = -\mathbf{U_B} \left(\left(\mathbf{R_B} - \mathbf{U_B}^T \mathbf{V_B}\right)^T \circ \mathbf{I_B}\right) + \lambda \mathbf{V_B} \quad (4)$$

where $\mathbf{B}$ represents the user-item rating set of the current minimum batch, and $\mathbf{R_B}$, $\mathbf{I_B}$ represent respectively the rating matrix, indicator matrix. $\mathbf{U_B}$ and $\mathbf{V_B}$ represent user-latent matrix and item-latent matrix of the current batch, respectively. $\mathbf{S_B}$ represents the social matrix corresponding to the current minimum batch of users, and $\mathbf{D_B}$ and $\mathbf{E_B}$ are matrices related to the social matrix. Note that $\mathbf{D_B}$, $\mathbf{S_B}$ and $\mathbf{E_B}$ are the private data of the social platform, while the other items are data owned by the rating platform.

By solving (3) and (4) for $\frac{\partial \mathcal{L}}{\partial \mathbf{U_B}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{V_B}}$, and iteratively applying the gradient descent algorithm, we can obtain the optimal solution for the objective function.

Equation (4) contains terms exclusively accessible to the rating platform P_0, allowing purely local computation. However, equation (3) incorporates three critical terms ($\mathbf{D_B}$, $\mathbf{S_B}$, and $\mathbf{E_B}$) that are private to the social platform P_1, creating a challenge to preserve privacy. The key problem is to securely evaluate the cross-platform products $\mathbf{U_B D_B}^T$, $\mathbf{U S_B}^T$, and $\mathbf{U_B E_B}^T$ required for SeSoRec optimization while preventing disclosure of sensitive data from P_1.

This security requirement translates into a secure matrix multiplication problem. Chen *et al.* [2] addressed this challenge through their secret-sharing based matrix multiplication protocol (SSMM) (Algorithm 1). As shown in Figure 1, the rating platform P_0 and the social platform P_1 securely construct a social recommendation system through the SeSoRec framework. The core of SeSoRec is that the two parties securely compute the cross-platform items $\mathbf{U_B D_B}^T$, $\mathbf{U S_B}^T$, and $\mathbf{U_B E_B}^T$ through SSMM.

3.2 Additive Secret Sharing

Secret sharing constitutes a fundamental building block of MPC protocols [22], with approaches based on secret sharing being widely used in privacy-preserving systems. Our work focuses on two-party additive secret sharing [6], which comprises the following three core algorithms (where $i \in \{0, 1\}$ denotes the party index):

- **Sharing**: To additively share a secret matrix $\mathbf{X} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$ (assume party P_i holds it), sharing protocol works as follows: P_i samples random matrix $\mathbf{R} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$ and sets $\langle \mathbf{X} \rangle_i = (\mathbf{X} - \mathbf{R}) \bmod 2^\ell$, and sends $\langle \mathbf{X} \rangle_{1-i} = \mathbf{R}$ to P_{1-i} where $\langle \mathbf{X} \rangle_i$ is P_i's secret share of $\mathbf{X}$ such that $\langle \mathbf{X} \rangle_0 + \langle \mathbf{X} \rangle_1 = \mathbf{X} \bmod 2^\ell$.

Algorithm 1: Secret Sharing-based Matrix Multiplication in [2]

Input : P_0 holds $\mathbf{A} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$, P_1 holds $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$
Output: P_i gets $\mathbf{C}_i$ s.t. $\mathbf{C}_0 + \mathbf{C}_1 = \mathbf{A} \times \mathbf{B}$

1 P_0 samples $\mathbf{E} \leftarrow \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$, and P_1 samples $\mathbf{F} \leftarrow \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$;
2 P_0 extracts even and odd columns from $\mathbf{E}$ to get $\mathbf{E}_{c_odd}$, $\mathbf{E}_{c_even} \in \mathbb{Z}_{2^\ell}^{d_1 \times \frac{d_2}{2}}$, and P_1 extracts odd and even rows from $\mathbf{F}$ to get $\mathbf{F}_{r_odd}$, $\mathbf{F}_{r_even} \in \mathbb{Z}_{2^\ell}^{\frac{d_2}{2} \times d_3}$;
3 P_0 computes $\mathbf{A}_0 = \mathbf{A} + \mathbf{E}, \mathbf{E}_0 = \mathbf{E}_{c_odd} + \mathbf{E}_{c_even}$ and sends $\mathbf{A}_0, \mathbf{E}_0$ to P_1. P_1 computes $\mathbf{B}_1 = \mathbf{F} - \mathbf{B}$, $\mathbf{F}_1 = \mathbf{F}_{r_even} - \mathbf{F}_{r_odd}$ and sends $\mathbf{B}_1, \mathbf{F}_1$ to P_0;
4 P_0 computes $\mathbf{C}_0 = (\mathbf{A} + 2 \cdot \mathbf{E}) \times \mathbf{B}_1 + (\mathbf{E}_{c_odd} + \mathbf{E}_0) \times \mathbf{F}_1$ locally, and P_1 computes $\mathbf{C}_1 = \mathbf{A}_0 \times (2 \cdot \mathbf{B} - \mathbf{F}) - \mathbf{E}_0 \times (\mathbf{F}_{r_even} + \mathbf{F}_1)$ locally;
5 **return** $(\mathbf{C}_0, \mathbf{C}_1)$ for P_0;

- **Addition and Multiplication**: Given two shared matrices $\langle \mathbf{X} \rangle_i$ and $\langle \mathbf{Y} \rangle_i$ holding by P_i, it is easy to non-interactively add the shared matrices by having P_i compute $\langle \mathbf{Z} \rangle_i = \langle \mathbf{X} \rangle_i + \langle \mathbf{Y} \rangle_i$. To multiply $\langle \mathbf{X} \rangle$ and $\langle \mathbf{Y} \rangle$, the parties need to exchange messages to compute $\langle \mathbf{Z} \rangle_0$ and $\langle \mathbf{Z} \rangle_1$ such that $\langle \mathbf{Z} \rangle_0 + \langle \mathbf{Z} \rangle_1 = (\langle \mathbf{X} \rangle_0 + \langle \mathbf{X} \rangle_1) \times (\langle \mathbf{Y} \rangle_0 + \langle \mathbf{Y} \rangle_1)$.
- **Reconstruction**: To reconstruct an additively shared value $\langle \mathbf{Z} \rangle$, reconstruction protocol works as follows. P_i sends $\langle \mathbf{Z} \rangle_i$ to P_{1-i} who computes $\mathbf{Z} = (\langle \mathbf{Z} \rangle_0 + \langle \mathbf{Z} \rangle_1) \bmod 2^\ell$.

In the following text, we omit the modular operation for simplicity.

3.3 Simulation-based Security Definition

Intuitively, a protocol is considered secure if it should be impossible for adversaries to learn any additional information beyond what is allowed by examining the transcript of messages received and the protocol's internal state [15]. Formally, we consider secure two-party computation against static, probabilistic polynomial-time, and semi-honest adversaries that corrupted parties have limited computing resources and time and must follow the protocol. Our security definition uses the real-ideal paradigm [15]. In the real world, two parties interact with each other according to the protocol Π. In the ideal world, two parties have access to a trusted third party that implements an ideal functionality $\mathcal{F}$. We say that Π securely realizes $\mathcal{F}$ if, for any adversary $\mathcal{A}$ in the real world, there exists a simulator $\mathcal{S}$ in the ideal world such that, given the corrupted party's input and output, $\mathcal{S}$ can generate an ideal view and output that are indistinguishable from the adversary's real view and output. The real view includes the corrupted party's input, random tape, and all received messages.

In this paper, we follow the semi-honest security definition of the ideal-real paradigm from [15].

Definition 3.1 (***Semi-Honest Security***). Let $\mathcal{F} = (\mathcal{F}_0, \mathcal{F}_1)$ be a functionality and let Π be a two-party protocol that computes $\mathcal{F}$. Π is said to securely compute $\mathcal{F}$ in the presence of static semi-honest adversaries if there exist probabilistic polynomial-time algorithm $\mathcal{S}_0$ and $\mathcal{S}_1$ such that:

$$\{(\mathcal{S}_0(1^\lambda, x, \mathcal{F}_0(x,y)), \mathcal{F}(x,y))\}_{x,y,\lambda} \overset{c}{\equiv} \{(\text{view}_0^\Pi(x,y,\lambda), \text{output}^\Pi(x,y,\lambda))\}_{x,y,\lambda}$$
$$\{(\mathcal{S}_1(1^\lambda, y, \mathcal{F}_1(x,y)), \mathcal{F}(x,y))\}_{x,y,\lambda} \overset{c}{\equiv} \{(\text{view}_1^\Pi(x,y,\lambda), \text{output}^\Pi(x,y,\lambda))\}_{x,y,\lambda}$$

where $x, y \in \{0,1\}^*$ such that $|x| = |y|$, and λ is security parameter, and $\overset{c}{\equiv}$ denotes computational indistinguishability.

4 Privacy Leakage Analysis with Two Corresponding Attacks

In this section, we first prove that SeSoRec's privacy leakage originates from key reuse in the SSMM protocol, making it fail to achieve semi-honest security under the real-ideal simulation paradigm. We then develop: (1) a general attack extracting intermediate values ($\mathbf{A}_{c_even} + \mathbf{A}_{c_odd}$ and $\mathbf{B}_{r_even} - \mathbf{B}_{r_odd}$) with two reconstruction algorithms—SMR for sparse matrices (validated on Epinions, FilmTrust and Douban) and GIR for images (tested on MNIST), and (2) a zero-padding attack directly exposing private inputs without reconstruction.

4.1 SeSoRec Privacy Leakage with Security Proof

While the authors of SeSoRec [2] acknowledge intermediate information leakage in SSMM, they assert these leaks do not compromise private inputs. Contrary to this claim, we demonstrate that such leakage indeed enables reconstruction of private data.

The root cause of SSMM information leakage is that it reuses the one-time pad key. In SSMM proposed by Chen *et al.* [2], platform P_0 (respectively P_1) samples a random matrix $\mathbf{E}$ (respectively $\mathbf{F}$) to mask the input matrix $\mathbf{A}$ (respectively $\mathbf{B}$), aiming to prevent the elements of the input matrix from being leaked. P_0 then mask the matrix $\mathbf{E}$ (respectively $\mathbf{F}$) by letting $\mathbf{E}_{c_odd} + \mathbf{E}_{c_even}$ (respectively $\mathbf{F}_{r_even} - \mathbf{F}_{r_odd}$) to ensure privacy. The method inherently utilizes a one-time pad [19, 20], recognized as a perfectly secure encryption scheme [11], to protect the privacy of inputs. For example, for P_0, the random matrix $\mathbf{E}$ serves as a one-time pad key to encrypt the input matrix $\mathbf{A}$, that is, $\mathbf{A}_0 = \mathbf{A} + \mathbf{E}$. In $\mathbf{E}_0 = \mathbf{E}_{c_odd} + \mathbf{E}_{c_even}$, the "even matrix" $\mathbf{E}_{c_even}$ can be considered as a one-time pad key for the "odd matrix" $\mathbf{E}_{c_odd}$. While this approach cleverly attempts to leverage the one-time pad for input matrix privacy, it inadvertently violates a fundamental rule of one-time pad usage — the key must never be reused, but that is not adhered to in this context.

The one-time pad encryption scheme requires that: 1) the key must be uniformly chosen from the key space, and 2) the key must never be reused to encrypt more than one message [1, 11]. Reusing a key compromises the entire encryption scheme's security [1]. However, in the secure matrix multiplication protocol proposed by Chen *et al.* [2], the one-time pad key $\mathbf{E}_{c_even}$ is utilized to encrypt both $\mathbf{A}_{c_even}$ and $\mathbf{E}_{c_odd}$, which means the protocol may no longer be secure. We present the security analysis as follows.

Security analysis of SSMM. We prove that SSMM does not satisfy the semi-honest security definition using the real-ideal simulation paradigm. Without loss of generality, we first consider the

case that P_0 is corrupted, and the case that P_1 is corrupted is similar. We need to show that a simulator $\mathcal{S}_0$ can generate an ideal view based on P_0's input and output, but the view is not indistinguishable from adversaries' real view. Formally, $\mathcal{S}_0$ is given P_1's input $\mathbf{A}$ and output $\mathbf{C}_1$ and works as follows (Note that simulator $\mathcal{S}_0$ cannot use $\mathbf{B}_{r_odd} - \mathbf{B}_{r_even}$ as prior knowledge like in [2] according to the security definition in Section 3.3):

(1) $\mathcal{S}_0$ choose a random matrix $\mathbf{E}' \leftarrow \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$ and extracts even and odd columns from $\mathbf{E}'$ to get $\mathbf{E}'_{c_odd}, \mathbf{E}'_{c_even} \in \mathbb{Z}_{2^\ell}^{d_1 \times \frac{d_2}{2}}$.
(2) $\mathcal{S}_0$ computes $\mathbf{A}'_0 = \mathbf{A} - \mathbf{E}', \mathbf{E}'_0 = \mathbf{E}'_{c_odd} + \mathbf{E}'_{c_even}$.
(3) $\mathcal{S}_0$ choose a random matrix $\mathbf{B}', \mathbf{F}' \leftarrow \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$, and computes $\mathbf{B}'_1 = \mathbf{B}' - \mathbf{F}', \mathbf{F}'_1 = \mathbf{F}'_{r_odd} - \mathbf{F}'_{r_even}$.
(4) $\mathcal{S}_0$ computes $\mathbf{C}'_0 = \mathbf{A} \times \mathbf{B}'_1 + \mathbf{E}'_{c_odd} \times \mathbf{F}'_1$.
(5) Finally, $\mathcal{S}_0$ outputs the ideal view $\mathcal{S}_0(1^\lambda, \mathbf{A}, \mathbf{C}_0) = (\mathbf{A}; \mathbf{E}'; \mathbf{B}'_1, \mathbf{F}'_1)$ and the ideal output $\mathcal{F}(\mathbf{A}, \mathbf{B}) = (\mathbf{C}'_0, \mathbf{C}_1)$.

The adversary's real view comprises the corrupted party's input, its random tape, and all messages it has received, which is denoted as $\text{view}_0^\Pi(\mathbf{A}, \mathbf{B}, \lambda) = (\mathbf{A}; \mathbf{E}; \mathbf{B}_1, \mathbf{F}_1)$. Note that in the real view, $\mathbf{B}_1$ and $\mathbf{F}_1$ satisfy the condition $\mathbf{F}_1 - (\mathbf{B}_{1,r_even} - \mathbf{B}_{1,r_odd}) = (\mathbf{B}_{r_odd} - \mathbf{B}_{r_even})$, whereas in the ideal view, $\mathbf{F}'_1 - (\mathbf{B}'_{1,r_even} - \mathbf{B}'_{1,r_odd}) = (\mathbf{B}'_{r_odd} - \mathbf{B}'_{r_even})$. Hence, the ideal view is indistinguishable from the real view if and only if $\mathbf{B}$ has the same distribution as $\mathbf{B}'$, meaning $\mathbf{B}$ must be a random matrix, but this contradicts that $\mathbf{B}$ is a specific matrix of social recommendation (e.g., matrix $\mathbf{B}$ is a sparse social matrix in social recommendation). Therefore, SSMM in Algorithm 1 is not secure against semi-honest adversaries according to the security definition (refer to Definition 3.1).

4.2 Two-Time Pad Attack

In this section, we introduce the Two-Time Pad Attack to obtain the leaked information in SSMM. Specifically, the party providing the left input of SSMM leaks the sum of the even and odd columns of its input matrix, while the party providing the right input leaks the difference between the even and odd rows of its input matrix. The notation in this section is consistent with that in Algorithm 1.

Attack toward Two-Time Pad. The one-time pad is secure only when each key is used once. Using the same key multiple times, a situation referred to as a Two-Time Pad, significantly compromises security [1]. It is evident that encrypting multiple messages with the same key results in considerable information leakage. In the SSMM described by Chen *et al.* [2], the matrix $\mathbf{A}$ contains private data from platform P_0 and $\mathbf{B}$ contains private data from platform P_1. Let $\mathbf{A}_{0,c_odd}$ and $\mathbf{A}_{0,c_even}$ be the matrix composed of all odd and even columns of $\mathbf{A}_0 = \mathbf{A} + \mathbf{E}$, and it holds that $\mathbf{A}_{0,c_odd} = \mathbf{A}_{c_odd} + \mathbf{E}_{c_odd}$, $\mathbf{A}_{0,c_even} = \mathbf{A}_{c_even} + \mathbf{E}_{c_even}$. Due to both $\mathbf{A}_{c_even}$ and $\mathbf{E}_{c_odd}$ are encrypted using the same key $\mathbf{E}_{c_even}$, the social platform P_1, corrupted by an adversary, could receive $\mathbf{A}_0$ and $\mathbf{E}_0$, and can compute: $(\mathbf{A}_{0,c_odd} + \mathbf{A}_{0,c_even}) - \mathbf{E}_0 = \mathbf{A}_{c_odd} + \mathbf{A}_{c_even}$. The adversary would learn $\mathbf{A}_{c_odd} + \mathbf{A}_{c_even}$, i.e., P_0' private information. Similarly, the platform P_0 corrupted by an adversary would obtain $\mathbf{B}_{r_even} - \mathbf{B}_{r_odd} = \mathbf{F_1} - (\mathbf{B}_{1,r_even} + \mathbf{B}_{1,r_odd})$, where $\mathbf{B}_{1,r_even}$ and $\mathbf{B}_{1,r_odd}$ denote the even and odd rows of $\mathbf{B}_1$. Moreover, if $\mathbf{A}_{c_odd}$ and $\mathbf{A}_{c_even}$ correspond to specific values (e.g., if $\mathbf{A}$ is a sparse matrix predominantly composed of zeros), then given $\mathbf{A}_{c_odd} + \mathbf{A}_{c_even}$, elements within $\mathbf{A}_{c_odd}$ or $\mathbf{A}_{c_even}$ may be revealed through cryptanalysis.

Algorithm 2: Sparse Matrix Reconstruction (SMR)

Input : Matrix $\mathbf{A} \in \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$
Output: Matrix $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{2d_2 \times d_3}$

```
Set a zero matrix B to be of size 2d_2 × d_3;
for i = 0; i < d_2; i = i + 1 do
    for j = 0; j < d_3; j = j + 1 do
        if A[i, j] = 0 then
            continue;
        else if A[i, j] = 1 then
            B[2i, j] = 0, B[2i + 1, j] = 1;
        else if A[i, j] = −1 then
            B[2i, j] = 1, B[2i + 1, j] = 0;
        else if A[i, j] = 2 then
            B[2i, j] = −1, B[2i + 1, j] = 1;
        else
            B[2i, j] = 1, B[2i + 1, j] = −1;
return matrix B;
```

We present two information reconstruction algorithms designed to reconstruct private information based on the leaked information and the characteristics of the input data in different scenarios. The Sparse Matrix Reconstruction algorithm (SMR, Algorithm 2) is applied when the input matrix is a sparse matrix primarily composed of 0s and 1s. It reconstructs the private input matrix by utilizing the difference between the even and odd columns of the leaked information. In the context of the SeSoRec framework, this algorithm can significantly analyze the private social information provided by the social platform.

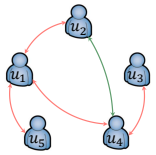
The Grayscale Image Reconstruction algorithm (GIR, Algorithm 3) is designed to reconstruct private information when the input matrix is a grayscale image. It utilizes the sum of even and odd rows or the difference between even and odd columns of the leaked information to recover the private data. When applied to the MNIST dataset, this algorithm can effectively recover the contour information of the images.

4.2.1 Experiments for SMR. As shown in Figure 1, in SSMM, the rating platform needs to provide the input as the latent user matrix $\mathbf{U}$, while the social platform needs to provide the input as the user's social matrix $\mathbf{S}$. Our focus is primarily on the leakage of private inputs from the social platform P_1. First, we run the SeSoRec framework on these datasets and use the Two-Time Pad Attack to obtain the leaked information $\mathbf{S}_{r_even} - \mathbf{S}_{r_odd}$ from P_1. Subsequently, we use Algorithm 2 to reconstruct the matrix based on the leaked information $\mathbf{S}_{r_even} - \mathbf{S}_{r_odd}$. Finally, we compare the reconstructed matrix to the privacy inputs of the social platform to determine how many privacy inputs can be inferred. Given that social matrices are typically sparse, with over 99.99% of the entries being zeros across all three datasets, guessing an entirely zero matrix would correctly predict over 99.99% of the data. Therefore, we focus our comparison only on the non-zero data.

Table 1: The Percentage of Information Leakage of Social Matrix

Dataset	FilmTrust	Epinions	Douban
Dimension ($N \times N$)	1,643	131,828	154,909
Non-Zero Ratio	99.35%	83.83%	77.14%

Notes: (1) "Dimension ($N \times N$)" denotes the dimensions of the social matrix (e.g., FilmTrust contains a 1,643 × 1,643 matrix). (2)"Non-zero ratio" indicates the ratio of the leaked non-zero items in the social matrix to the total non-zero items.



(a) Example of Social Network

	u_1	u_2	u_3	u_4	u_5
u_1	0	1	0	1	1
u_2	1	0	0	1	0
u_3	0	0	0	1	0
u_4	1	1	1	0	0
u_5	1	0	0	0	0

(b) Corresponding Adjacency Matrix

	u_1	u_2	u_3	u_4	u_5
u_1	0	1	0	1	1
u_2	1	0	0	0	0
u_3	0	0	0	1	0
u_4	1	0	1	0	0
u_5	1	0	0	0	0

(c) Reconstructed Adjacency Matrix

Figure 2: Example of Social Network Privacy Leakage.

We execute the SeSoRec [2] framework on three real-world public datasets: FilmTrust [9], Epinions [10], and Douban [14]. Table 1 presents our experimental results where "Dimension" indicates the dimensions of the private input social matrix for the social platform P_1 and "Non-Zero Ratio" denotes the percentage of non-zero elements within the social matrix that are correctly guessed.

During these tests, we extract leaked information and conduct a reconstruction. By comparing the reconstructed information with the original input information in these datasets, we determine that it is possible to reveal over 99.35%, 83.83%, and 77.14% of the private data on the respective datasets. This suggests that while executing SeSoRec, the majority of the information from the social platform can be readily accessed by the rating platform, thereby compromising the security of the framework.

Figure 2 demonstrates social network reconstruction via adjacency matrix analysis, showing: (a) the original platform-provided social matrix with red edges indicating successfully recovered connections, (b) its corresponding network graph, and (c) our reconstructed private matrix. The experiment successfully recovered 8 of 10 ground-truth non-zero entries, achieving 80% reconstruction accuracy ($8/10 \times 100\%$), where red edges in (a) correspond to the reconstructed connections in (c).

4.2.2 Experiments for GIR. In the previous section, we conducted experiments solely on the information leakage of the social platform providing the right input to SSMM within the SeSoRec framework. In reality, the information leaked by the parties providing the left and right inputs to differs. To visually demonstrate the privacy leakage for parties providing both left and right inputs, we conducted experiments on the MNIST dataset. As analyzed in Section 4.2, during the execution of SSMM, the party on the left side of the matrix multiplication leaks the sum of the even and odd columns of the input matrix, while the party on the right side leaks the difference between the even and odd rows of the input matrix. First, we use the Two-Time Pad Attack to obtain the leaked information. Then, we use this leaked information as input to Algorithm 3 to

Algorithm 3: Grayscale Image Reconstruction (GIR)

Input : Matrix $\mathbf{A} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$
Number flag $\in \{0, 1\}$
Output : If flag = 0, matrix $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{d_1 \times 2d_2}$
Otherwise matrix $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{2d_1 \times d_2}$

```
if flag = 0 then
    Set B as zero matrix of size d_1 × 2d_2;
else
    Set B as zero matrix of size 2d_1 × d_2;
for i = 0; i < d_1; i = i + 1 do
    for j = 0; j < d_2; j = j + 1 do
        if A[i, j] = 0 then
            continue;
        else if flag = 0 then
            B[i, 2j] = 0, B[i, 2j + 1] = A[i, j];
        else if flag = 1 and A[i, j] < 0 then
            B[2i, j] = −A[i, j], B[2i + 1, j] = 0;
        else
            B[2i, j] = 0, B[2i + 1, j] = A[i, j];
return matrix B;
```

reconstruct a matrix. Finally, we display the reconstructed matrix as a binary image, visualizing any matrix as a grayscale image by mapping each entry of the matrix to a block (i.e., mapping 0 to a white block and 1 to a black block).

In Algorithm 3, when flag is set to 0, it indicates that the leaked information is the sum of the even and odd columns of the matrix; when set to 1, it indicates that the leaked information is the difference between the even and odd rows.

We conducted experiments on the MNIST dataset. Figure 3 presents the binary images of the matrices reconstructed using leaked information. In Figure 3, columns 1, 4, 7, 10, and 13 display five randomly selected original images of digits $1 \sim 9$. Columns 2, 5, 8, 11, and 14 show the matrices reconstructed based on the sum of even and odd columns of information leaked after the SSMM multiplication of the original images. Columns 3, 6, 9, 12, and 15 illustrate the matrices reconstructed from the difference between even and odd rows of information leaked following the SSMM multiplication of the original images.

The experiments demonstrate that binary images of matrices reconstructed from leaked information in the MNIST dataset reveal the data of the original images, although the clarity of the reconstructed images varies depending on the leaked information.

4.3 Zero-Padding Attack

We have also identified information leakage under specific circumstances. Specifically, when the sum of columns in the left input matrix and the sum of rows in the right input matrix are not even, SSMM suggests padding with rows or columns filled with zeros. However, our analysis indicates that this zero-padding may lead to the leakage of a one-time pad key, compromising the security of

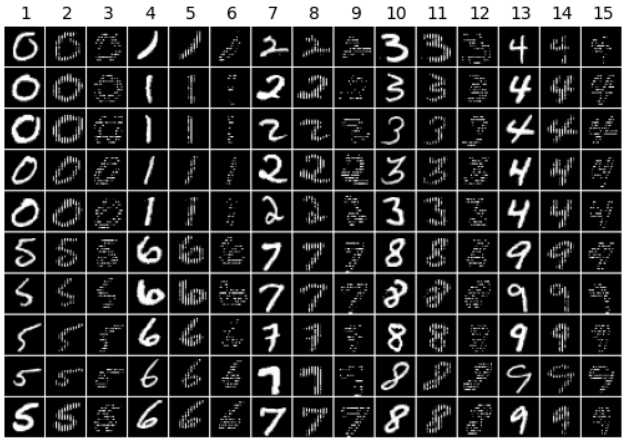


Figure 3: Privacy Leakage on MNIST Caused by SSMM

the scheme. We demonstrate that this approach directly leaks the last column or last row of the input matrix. Below, we present the Zero-Padding Attack, which demonstrates how to obtain the last column or last row of the input matrix from SSMM.

Attack toward Zero-Padding. Assume rating platform P_0 holds input matrix $\mathbf{A} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$, and rating platform P_1 holds input matrix $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$. Without loss of generality, let us assume that the rating platform P_0 is the honest party, and the social platform P_1 is the corrupted party. When d_2 is an odd number, both P_0 and P_1 agree to perform zero-padding; specifically, P_0 adds a zero column at the end of matrix $\mathbf{A}$, and P_1 adds a zero row at the end of matrix $\mathbf{B}$. From our previous analysis, we understand that when the adversary controls P_1, the sum of the odd and even columns of matrix $\mathbf{A}$ leaked by P_0 and the difference between the even and odd rows of matrix $\mathbf{B}$ leaked by P_1 can be exploited. Consider the value of the last column of $\mathbf{A}$, denoted as $\mathbf{A}[:d_2-1]$, plus the zero column, resulting in $\mathbf{a} = \mathbf{A}[:d_2-1] + \mathbf{A}[:d_2]$. Since $\mathbf{A}[:d_2]$ is a zero column and $\mathbf{a}$ is a part of the leaked information (sum of even and odd rows), $\mathbf{A}[:d_2-1]$ is effectively leaked. Similarly, the value of the last row of $\mathbf{B}$ is also directly leaked.

Figure 4 illustrates a toy example of two attacks. In this figure, the solid circular line represents information leakage caused by matrix sparsity, where knowing the positions of zeros in matrix $\mathbf{A}$ reveals the specific values leaked, and the dotted circular line illustrates information leakage due to zero-padding.

5 Countermeasure

5.1 Privacy-Preserving Matrix Multiplication

To address the above privacy leakage issues in [2], we present a privacy-preserving matrix multiplication (PPMM) protocol based on secret sharing and provide formal security proof. We adopt the offline-online computation diagram [4], and the matrix multiplication triples technique [17]. Specifically, given matrix $\mathbf{A} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$ holding by platform P_0 and $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$ holding by platform P_1, parties P_0 and P_1 invoke a matrix multiplication triples generation scheme [17, 21] to precompute matrix multiplication triples

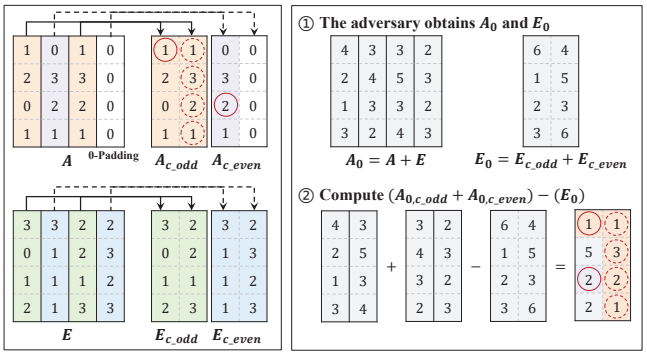


Figure 4: Toy Example of Attack toward Two-Time Pad and Zero-Padding.

Algorithm 4: Privacy-Preserving Matrix Multiplication (PPMM)

Input : P_0 holds $\mathbf{A} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$
P_1 holds $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$
Output: P_i gets $\langle \mathbf{C} \rangle_i$ s.t. $\langle \mathbf{C} \rangle_0 + \langle \mathbf{C} \rangle_1 = \mathbf{A} \times \mathbf{B}$

1 **Offline Phase** *Precompute matrix multiplication triples*
2 P_0 and P_1 invoke $\mathcal{F}_{\mathsf{MMT}}$ to generate a matrix multiplication triple $(\langle \mathbf{E} \rangle, \langle \mathbf{F} \rangle, \langle \mathbf{EF} \rangle)$ where P_i gets $\langle \mathbf{E} \rangle_i, \langle \mathbf{F} \rangle_i, \langle \mathbf{EF} \rangle_i$ s.t. $(\langle \mathbf{E} \rangle_0 + \langle \mathbf{E} \rangle_1) \times (\langle \mathbf{F} \rangle_0 + \langle \mathbf{F} \rangle_1) = (\langle \mathbf{EF} \rangle_0 + \langle \mathbf{EF} \rangle_1)$;
3 **Online Phase** *Perform matrix multiplication using triples*
4 P_0 computes $\langle \hat{\mathbf{A}} \rangle_0 = \mathbf{A} - \langle \mathbf{E} \rangle_0$, and sends $\langle \hat{\mathbf{A}} \rangle_0, \langle \mathbf{F} \rangle_0$ to P_1. P_1 computes $\langle \hat{\mathbf{B}} \rangle_1 = \mathbf{B} - \langle \mathbf{F} \rangle_1$, and sends $\langle \hat{\mathbf{B}} \rangle_1, \langle \mathbf{E} \rangle_1$ to P_0.
5 P_i computes $\hat{\mathbf{A}} = \langle \hat{\mathbf{A}} \rangle_0 - \langle \mathbf{E} \rangle_1$ and $\hat{\mathbf{B}} = \langle \hat{\mathbf{B}} \rangle_1 - \langle \mathbf{F} \rangle_0$ locally.
6 P_0 computes $\langle \mathbf{C} \rangle_0 = \langle \mathbf{E} \rangle_0 \times \hat{\mathbf{B}} + \hat{\mathbf{A}} \times \langle \mathbf{F} \rangle_0 + \langle \mathbf{EF} \rangle_0$ locally, and P_1 computes $\langle \mathbf{C} \rangle_1 = \hat{\mathbf{A}} \times \hat{\mathbf{B}} + \langle \mathbf{E} \rangle_1 \times \hat{\mathbf{B}} + \hat{\mathbf{A}} \times \langle \mathbf{F} \rangle_1 + \langle \mathbf{EF} \rangle_1$ locally.

in the offline phase, and utilize these triples to perform matrix multiplication in the online phase. Finally, P_i gets $\langle \mathbf{C} \rangle_i$ such that $\langle \mathbf{C} \rangle_0 + \langle \mathbf{C} \rangle_1 = \mathbf{A} \times \mathbf{B}$. Let $\mathcal{F}_{\mathsf{MMT}}$ denote the matrix multiplication triples generation functionality and $\mathcal{F}_{\mathsf{MatMul}}$ denote the privacy-preserving matrix multiplication functionality. Functionality $\mathcal{F}_{\mathsf{MMT}}$ can be implemented by using the matrix multiplication triples generation scheme [17, 21], and functionality $\mathcal{F}_{\mathsf{MatMul}}$ is implemented by the protocol Π_{MatMul}.

Algorithm 4 illustrates the privacy-preserving matrix multiplication protocol Π_{MatMul}, which requires an online communication of $2\ell(d_1d_2 + d_2d_3)$ bits in 1 rounds. Note that the parameter d_2 (i.e., the number of columns in $\mathbf{A}$ and the number of rows in $\mathbf{B}$) must be a positive even number in SSMM, but our protocol supports arbitrary matrix dimensions.

Efficiency Analysis. We compare the communication complexity of secret sharing-based matrix multiplication in [2] and our privacy-preserving matrix multiplication in Table 2.

Table 2: Comparison of Communication Complexity of Privacy-Preserving Matrix Multiplication between $\mathbf{A} \in \mathbb{Z}_{2^\ell}^{d_1 \times d_2}$ and $\mathbf{B} \in \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$

Protocol	# of triples	Comm. (bits)	Assumption
SSMM [2]	0	$\frac{3}{2}\ell(d_1d_2 + d_2d_3)$	Allow information leakage
PPMM (Ours)	$d_1d_2 + d_2d_3$	$2\ell(d_1d_2 + d_2d_3)$	N/A

Notes: "# of triples" denotes the number of multiplication triples required by these protocols.

Table 3: Comparison of Communication Complexity of Privacy-Preserving Social Recommendation Framework

Framework	# of triples	Comm. (bits)	Privacy Protection
SeSoRec [2]	0	$\frac{3}{2}\ell(d_1d_2 + 3d_2d_3)$	×
SeSoRec-Sec (Ours)	$d_1d_2 + 3d_2d_3$	$2\ell(d_1d_2 + 3d_2d_3)$	✓

Notes: "# of triples" denotes the number of multiplication triples required by these frameworks.

Although our protocol has slightly higher communication overhead, it does not require any assumptions. Additionally, our protocol can benefit from vectorization techniques and utilize correlated matrix multiplication techniques for communication optimization in social recommendation settings. Consequently, it is capable of handling privacy-preserving social recommendations on a large scale.

Security Analysis. The correctness of the protocol is oblivious, and Theorem 5.1 captures the security of Algorithm 4. We present the security proof of Theorem 5.1 as follows.

Theorem 5.1. *In the $\mathcal{F}_{\mathsf{MMT}}$-hybrid model, Π_{MatMul} in Algorithm 4 securely computes the functionality $\mathcal{F}_{\mathsf{MatMul}}$ with computational security against semi-honest adversaries.*

Proof. Without loss of generality, let P_0 be the party corrupted by adversaries, and the proof is similar when P_1 is the corrupted party. For a corrupted party P_0, we need to construct a simulator $\mathcal{S}_0$ who can generate a simulated view that is indistinguishable from adversaries' view in the real world. Formally, $\mathcal{S}_0$ is given P_0's input $\mathbf{A}$ and output $\langle \mathbf{C} \rangle_0$ and work as follows:

(1) $\mathcal{S}_0$ makes a black-box access to $\mathcal{F}_{\mathsf{MMT}}$ to generate a matrix multiplication triple $(\langle \mathbf{E}' \rangle_0, \langle \mathbf{F}' \rangle_0, \langle \mathbf{E}'\mathbf{F}' \rangle_0; \langle \mathbf{E}' \rangle_1, \langle \mathbf{F}' \rangle_1, \langle \mathbf{E}'\mathbf{F}' \rangle_1)$.

(2) $\mathcal{S}_0$ samples $\mathbf{B}' \leftarrow \mathbb{Z}_{2^\ell}^{d_2 \times d_3}$ and computes $\langle \hat{\mathbf{A}}' \rangle_0 = \mathbf{A} - \langle \mathbf{E}' \rangle_0, \langle \hat{\mathbf{B}}' \rangle_1 = \mathbf{B}' - \langle \mathbf{F}' \rangle_1$.

(3) $\mathcal{S}_0$ computes $\hat{\mathbf{A}}' = \langle \hat{\mathbf{A}}' \rangle_0 - \langle \mathbf{E}' \rangle_1$ and $\hat{\mathbf{B}}' = \langle \hat{\mathbf{B}}' \rangle_1 - \langle \mathbf{F}' \rangle_0$.

(4) $\mathcal{S}_0$ holds $\langle \mathbf{C} \rangle_0$ and computes $\langle \hat{\mathbf{C}} \rangle_1 = \hat{\mathbf{A}}' \times \hat{\mathbf{B}}' + \langle \mathbf{E}' \rangle_1 \times \hat{\mathbf{B}}' + \hat{\mathbf{A}}' \times \langle \mathbf{F}' \rangle_1 + \langle \mathbf{E}'\mathbf{F}' \rangle_1$

(5) Finally, $\mathcal{S}_0$ outputs the ideal view $\mathcal{S}_0(1^\lambda, \mathbf{A}, \langle \mathbf{C} \rangle_0) = (\mathbf{A}; \langle \hat{\mathbf{B}}' \rangle_1, \langle \mathbf{E}' \rangle_1)$ and outputs $\mathcal{F}(\mathbf{A}, \mathbf{B}) = (\langle \mathbf{C} \rangle_0, \langle \mathbf{C}' \rangle_1)$.

We know the real view is $\mathrm{view}_0^\Pi(\mathbf{A}, \mathbf{B}, \lambda) = (\mathbf{A}; \langle \hat{\mathbf{B}} \rangle_1, \langle \mathbf{E} \rangle_1)$ and the output is $\mathrm{output}^\Pi(\mathbf{A}, \mathbf{B}, \lambda) = (\langle \mathbf{C} \rangle_0, \langle \mathbf{C} \rangle_1)$, and is indistinguishable from the ideal view and output since $\langle \hat{\mathbf{B}}' \rangle_1, \langle \mathbf{E}' \rangle_1, \langle \mathbf{C}' \rangle_1$ have the same distribution as $\langle \hat{\mathbf{B}} \rangle_1, \langle \mathbf{E} \rangle_1, \langle \mathbf{C} \rangle_1$. Therefore, Π_{MatMul} in Algorithm 4 securely computes the functionality $\mathcal{F}_{\mathsf{MatMul}}$ with computational security against semi-honest adversaries. □

5.2 Privacy-Preserving Social Recommendation

Based on the privacy-preserving matrix multiplication protocol in Algorithm 4, we can build the privacy-preserving social recommendation framework. In SeSoRec, platforms P_0 and P_1 need to securely calculate the cross-platform products $\mathbf{U_B} \times \mathbf{D_B}^T$, $\mathbf{U} \times \mathbf{S_B}^T$, and $\mathbf{U_B} \times \mathbf{E_B}^T$, while other operations can be calculated locally. The naive implementation can be realized by invoking the PPMM protocol in Algorithm 4 three times, resulting in a total online communication complexity of $6\ell(d_1d_2 + d_2d_3)$ bits.

To further reduce communication costs, we observe that the matrix multiplications $\mathbf{U_B} \times \mathbf{D_B}^T$, $\mathbf{U} \times \mathbf{S_B}^T$, and $\mathbf{U_B} \times \mathbf{E_B}^T$ share the same operand $\mathbf{U}$ or its submatrix $\mathbf{U_B}$, which is selected using indices in B. Leveraging this observation, we can adopt the correlated matrix multiplication technique proposed in [12, 17] to securely perform these matrix multiplications, instead of independently invoking the PPMM protocol. Specifically, in our PPMM protocol in Algorithm 4, two independent matrix multiplication triples $(\langle \mathbf{E}_1 \rangle, \langle \mathbf{F}_1 \rangle, \langle \mathbf{E}_1\mathbf{F}_1 \rangle)$, $(\langle \mathbf{E}_2 \rangle, \langle \mathbf{F}_2 \rangle, \langle \mathbf{E}_2\mathbf{F}_2 \rangle)$ and $(\langle \mathbf{E}_3 \rangle, \langle \mathbf{F}_3 \rangle, \langle \mathbf{E}_3\mathbf{F}_3 \rangle)$ are required, where $(\langle \mathbf{E}_j \rangle_0 + \langle \mathbf{E}_j \rangle_1) \times (\langle \mathbf{F}_j \rangle_0 + \langle \mathbf{F}_j \rangle_1) = (\langle \mathbf{E}_j\mathbf{F}_j \rangle_0 + \langle \mathbf{E}_j\mathbf{F}_j \rangle_1)$ for $j = \{1, 2, 3\}$. Instead, we utilize correlated matrix multiplication triples [12], that is, $(\langle \mathbf{E} \rangle, \langle \mathbf{F}_1 \rangle, \langle \mathbf{EF}_1 \rangle)$, $(\langle \mathbf{E} \rangle, \langle \mathbf{F}_2 \rangle, \langle \mathbf{EF}_2 \rangle)$ and $(\langle \mathbf{E} \rangle, \langle \mathbf{F}_3 \rangle, \langle \mathbf{EF}_3 \rangle)$ where $(\langle \mathbf{E} \rangle_0 + \langle \mathbf{E} \rangle_1) \times (\langle \mathbf{F}_j \rangle_0 + \langle \mathbf{F}_j \rangle_1) = (\langle \mathbf{EF}_j \rangle_0 + \langle \mathbf{EF}_j \rangle_1)$ for $j = \{1, 2, 3\}$. By leveraging correlated matrix multiplication triples, the online communication cost is reduced from $6\ell(d_1d_2 + d_2d_3)$ bits to $2\ell(d_1d_2 + 3d_2d_3)$ bits, while SeSoRec [2] requires $\frac{3}{2}\ell(d_1d_2 + 3d_2d_3)$ bits using the same correlated matrix multiplication triples technique.

Efficiency Analysis. We compare the communication complexity of SeSoRec [2] and our SeSoRec-Sec method in Table 3. Although our method incurs slightly higher communication overhead, it ensures the privacy of social recommendations, whereas SeSoRec [2] exposes confidential information in social recommendation.

Security Analysis. The correctness of our privacy-preserving social recommendation framework is evident. The security of our framework follows from the security of our PPMM protocol. We omit the proof due to the page limit.

5.3 Experimental Evaluation of countermeasure

In this section, we evaluate the effectiveness of the PPMM protocol from two perspectives.

First, we validate the security guarantees of PPMM. As proven in Theorem 5.1 (Section 5.1), the PPMM protocol achieves semi-honest security and prevents any information leakage. Additional attack experiments (consistent with Section 4.2) confirm that the Two-Time Pad attack remains ineffective against PPMM.

Second, we reproduce the SeSoRec framework by replacing insecure SSMM with our PPMM. Experimental results on both FilmTrust and Epinions datasets demonstrate that our solution maintains model performance while providing enhanced security.

Security. We applied the same experimental method to the PPMM protocol, using the Two-Time Pad Attack to obtain 'leakage

information' (which is actually uniformly random and does not disclose any real information), followed by the two information reconstruction algorithms to attempt to recover private inputs. For the three common datasets—FilmTrust, Epinions, and Douban—the SMR algorithm recovered 0% of the private input information. In the experiments on MNIST, Figure 5 visualizes the results: columns 1, 4, 7, 10, and 13 show five randomly selected original images of digits 1-9; columns 2, 5, 8, 11, and 14 display the reconstructed left input information using the PPMM protocol and the Two-Time Pad with the GIR algorithm; columns 3, 6, 9, 12, and 15 show the reconstructed right input information. It is evident that the original image contours are no longer visually discernible. Although the images do not appear completely uniform, this is due to the limitations of the Information Reconstruction algorithm. The Two-Time Pad Attack is ineffective against our countermeasure.

Efficiency. Since the original SeSoRec implementation was unavailable, we faithfully reconstructed the framework to evaluate our PPMM protocol's efficiency. Our experiments on FilmTrust and Epinions datasets compared the original SeSoRec with our modified version (SeSoRec-Sec using PPMM instead of SSMM), employing two standard metrics: Root Mean Square Error (RMSE) and Normalized Discounted Cumulative Gain (NDCG@k). RMSE measures the accuracy of the predicted ratings and is calculated as:

$$\text{RMSE} = \sqrt{\frac{1}{T}\sum_{i,j}(R_{ij} - \hat{R}_{ij})^2} \tag{5}$$

where R_{ij} is the true rating provided by user i for item j, $\hat{R}_{ij}$ denotes the predicted rating, and T represents the total number of ratings in the test set. Lower RMSE values indicate better prediction accuracy.

NDCG@k assesses the quality of the top-k recommendation list. In our experiments, we use NDCG@10, defined as:

$$\text{NDCG@10} = \frac{\text{DCG@10}}{\text{IDCG@10}} \tag{6}$$

where DCG@10 (Discounted Cumulative Gain at rank 10) is computed as:

$$\text{DCG@10} = \sum_{i=1}^{10} \frac{2^{r_i} - 1}{\log_2(i+1)} \tag{7}$$

where r_i is the true rating (on a scale of [1, 5]) of the item at position i in the recommended list, and IDCG@10 represents the maximum possible DCG@10 score achieved by the ideal ranking. Higher NDCG@10 values signify better ranking performance.

For hyperparameter settings, we maintained identical configurations to those in SeSoRec to ensure fair comparison.

The comparative results in Table 4 show that SeSoRec-Sec maintains equivalent recommendation accuracy to the original SeSoRec framework. The observed marginal differences originate from SSMM protocol's requirement for zero-padding when processing matrices with odd dimensions (where either the left matrix's column count or right matrix's row count is odd).

6 Conclusion

In this paper, we conduct a detailed cryptanalysis of SeSoRec and demonstrate its insecurity from a provable security perspective. We propose the Two-Time Pad Attack and the Zero-Padding Attack to extract leaked information from SeSoRec, along with two information reconstruction methods to recover private data. We perform experiments on multiple datasets to quantify the leakage by revealing the private input matrices and showing the percentage of original inputs recovered. Additionally, we conduct experiments on the MNIST image dataset to visually illustrate this leakage. Experimental results across various datasets reveal significant leakage of private inputs, rendering the SeSoRec framework unsuitable for practical applications. The communication overhead of the PPMM protocol is higher than that of SSMM, and in future work, we aim to improve the PPMM protocol to further reduce this overhead.

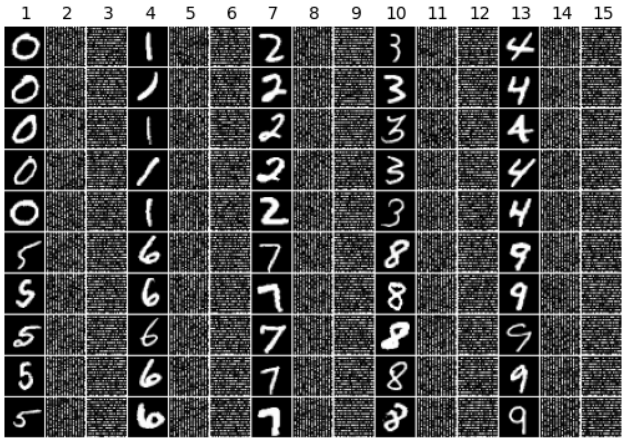


Figure 5: Countermeasure against privacy leakage

Table 4: Performance Comparison Between SeSoRec and SeSoRec-Sec (Ours)

Dataset	Metric	SeSoRec [2]	SeSoRec-Sec	Change
FilmTrust	RMSE	2.6824	2.6779	↓ 0.17%
	NDCG	0.5342	0.5340	↓ 0.04%
Epinions	RMSE	2.9741	3.0262	↑ 1.75%
	NDCG	0.9352	0.9323	↓ 0.31%

Acknowledgments

This project is supported by Shenzhen Science and Technology Major Special Project (KJZD20230923114608017), National Natural Science Foundation of China (72442029), HKU-SCF FinTech Academy and Shenzhen-Hong Kong-Macao Science and Technology Plan Project (Category C Project: SGDX202108 23103537030) and Theme-based Research Scheme T35-710/20-R, Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies (2022B1212010005).

Data Availability

The code and data are available at https://github.com/c-doubley/SeSoRec-Sec-Public.git under MIT License.

References

[1] Dan Boneh and Victor Shoup. 2023. A graduate course in applied cryptography. *Draft 0.6* (2023).

[2] Chaochao Chen, Liang Li, Bingzhe Wu, Cheng Hong, Li Wang, and Jun Zhou. 2020. Secure Social Recommendation based on Secret Sharing. In *ECAI 2020*. IOS Press, 506–512.

[3] Jinming Cui, Chaochao Chen, Lingjuan Lyu, Carl Yang, and Wang Li. 2021. Exploiting data sparsity in secure cross-platform social recommendation. *Advances in Neural Information Processing Systems* 34 (2021), 10524–10534.

[4] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. 2012. Multiparty computation from somewhat homomorphic encryption. In *Advances in Cryptology–CRYPTO 2012*. Springer, 643–662.

[5] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.

[6] Daniel Escudero. 2022. An introduction to secret-sharing-based secure multiparty computation. *Cryptology ePrint Archive* (2022).

[7] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 169–178.

[8] Oded Goldreich, Silvio Micali, and Avi Wigderson. 1987. How to play any mental game. In *Proceedings of the Nineteenth ACM Symp. on Theory of Computing, STOC*. ACM, 218–229.

[9] Guibing Guo, Jie Zhang, and Neil Yorke-Smith. 2013. A novel Bayesian similarity measure for recommender systems. In *Proceedings of the Twenty-Third international joint conference on Artificial Intelligence*. 2619–2625.

[10] Reyhani Masoud Hamedani, Irfan Ali, Jiwon Hong, and Sang-Wook Kim. 2021. TrustRec: An effective approach to exploit implicit trust and distrust relationships along with explicitones for accurate recommendations. *Computer Science and Information Systems* 18, 1 (2021), 93–114.

[11] Jonathan Katz and Yehuda Lindell. 2021. *Introduction to Modern Cryptography (3rd edition)*. CRC Press.

[12] Mahimna Kelkar, Phi Hung Le, Mariana Raykova, and Karn Seth. 2022. Secure poisson regression. In *31st USENIX Security Symposium (USENIX Security 22)*. 791–808.

[13] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.

[14] Jérôme Kunegis. 2013. Konect: the koblenz network collection. In *Proceedings of the 22nd international conference on world wide web*. 1343–1350.

[15] Yehuda Lindell. 2017. How to simulate it–a tutorial on the simulation proof technique. *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich* (2017), 277–346.

[16] Hai Liu, Chao Zheng, Duantengchuan Li, Xiaoxuan Shen, Ke Lin, Jiazhang Wang, Zhen Zhang, Zhaoli Zhang, and Neal N Xiong. 2021. EDMF: Efficient deep matrix factorization with review feature learning for industrial recommender system. *IEEE Transactions on Industrial Informatics* 18, 7 (2021), 4361–4371.

[17] Payman Mohassel and Yupeng Zhang. 2017. Secureml: A system for scalable privacy-preserving machine learning. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 19–38.

[18] Valeria Nikolaenko, Stratis Ioannidis, Udi Weinsberg, Marc Joye, Nina Taft, and Dan Boneh. 2013. Privacy-preserving matrix factorization. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 801–812.

[19] Claude Elwood Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal* 27, 3 (1948), 379–423.

[20] Gilbert S Vernam. 1926. Cipher printing telegraph systems: For secret wire and radio telegraphic communications. *Journal of the AIEE* 45, 2 (1926), 109–115.

[21] Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. 2020. Ferret: Fast extension for correlated OT with small communication. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 1607–1626.

[22] Andrew Chi-Chih Yao. 1986. How to Generate and Exchange Secrets. In *27th Annual Symposium on Foundations of Computer Science (sfcs 1986)*. IEEE, 162–167.